QuipKeys: Decoding Imprecise Physical-Keyboard Typing

Extended report: system, simulated and real-typing evaluation,
and what a neural language model does and does not add

Hasnat

https://github.com/codinghasnat/quipkeys

November 2026

Abstract

Phone keyboards decode where a finger probably meant to land; physical keyboards still rely on conservative dictionary autocorrect. QuipKeys is an offline decoder for physical-keyboard typing whose goal is that a person can type fast and imprecisely, without looking at the keys and without backspace, and still get the words they meant.

It combines four parts:

- an error model of keyboard slips;
- a weighted edit-distance search over a dictionary trie;
- an interpolated trigram language model built from 1.4 billion words;
- a sliding-window lattice decoder, whose confidence in a correction is that correction's posterior probability summed over all decoding paths.

Safety mechanisms hold false corrections to about one in 750 correctly typed words. It also repairs misplaced spaces, whole-hand shifts, Shift-key slips and accidental Caps Lock, and switches automatically to a rough-aim error model when the typist is sloppy.

On held-out simulated typing it fixes 77.3% of mistyped words at moderate error rates (a frequency-only spellchecker: 53.5%), and lifts word accuracy for blind typing (10% of letters wrong) from 61.7% to 89.0%. On 33 passages of real, blind, no-backspace typing by one participant at a median of 85 WPM, words typed exactly right rose from 75.4% to 91.5%, and 92.5% with a fitted personal error model. The real data overturned two simulator assumptions, and made keyboard geometry irrelevant for this typist.

Finally, a 23M-parameter neural language model, trained in three hours on a consumer GPU to a perplexity of 59, gave at most +0.3 points. The errors that remain lack *information*, not model capacity, and we discuss what could supply it.

Contents

1 Introduction **2**

2 Related Work **3**

3 System **3**

3.1 Formulation . . . 3
3.2 Error model and candidate search . . . 4
3.3 Language model . . . 4
3.4 Sliding-window lattice decoder . . . 4
3.5 Safety mechanisms . . . 5
3.6 Errors beyond letters . . . 6
3.7 Adapting to sloppiness . . . 6

4 Evaluation Methodology 6

5 Results on Simulated Typing 7

6 Real Typing 8

7 A Neural Language Model 10

8 What Remains, and Lessons 11

9 Limitations 12

10 Future Work: More Information, Not More Intelligence 12

11 Conclusion 13

1 Introduction

Typing accurately is slow. A large share of everyday keystrokes are spent on correction, even for fast typists [3]: noticing an error, backspacing over it, retyping. Watching for errors also pulls attention from what one is writing.

Touchscreens solved a harder version of the problem. A finger on glass has no key boundaries, so phone keyboards treat each tap as noisy evidence and decode the intended words with a language model [6, 10, 16]. Physical keyboards never made that transition. Desktop autocorrect, in the operating system or the word processor, is still a cautious dictionary spellchecker: it fixes `teh`, but it does not reason about which key a finger meant, rarely uses context, and does not repair spacing.

The north star. The target user does not care where their fingers land. There is intent, but each press lands only roughly near the intended key, and they never look down or backspace. The metric that matters is *effective words per minute*: correct words per minute after all corrections. Figure 1 shows the kind of input the system must handle.

Typed: `Hellon ice to mee tyou i hopw ypur doimg wel. w NEED TO FINSIH THIS VEFORE FRIDAY`

↓

QuipKeys: Hello nice to meet you I hope your doing well. we need to finish this before Friday

Figure 1: Spacing slips (`Hellon ice`, `mee tyou`), adjacent-key slips (`hopw`, `ypur`), a missing letter (`wel`) and accidental Caps Lock are all repaired. `your` stays: it is a real word, and the grammar fix (*you're*) is out of scope.

Contributions.

1. A complete offline decoder for physical keyboards, in Rust, at about 1 ms per word (§3).
2. Safety mechanisms against the failure that makes people turn correction off: changing what they meant (§3.5).
3. Repairs that word-level spellcheckers cannot make: spaces, hand shifts, Shift slips, Caps Lock (§3.6).

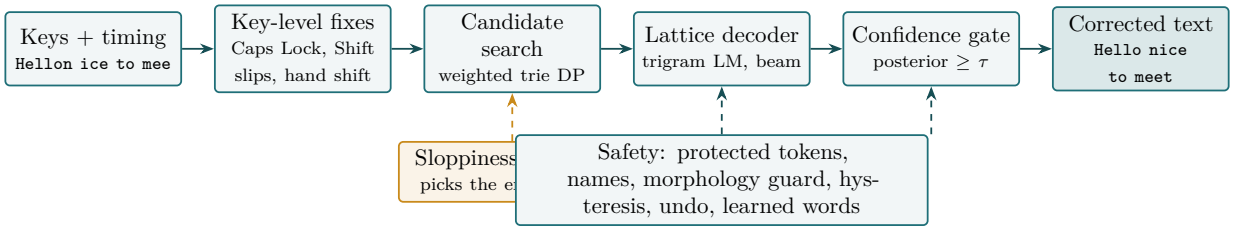


Figure 2: The QuipKeys pipeline. Each chunk (the text between spaces) is decided when its space is pressed, so corrections are immediate. The last four chunks are then re-decoded jointly, so later words can fix earlier ones.

4. A five-level benchmark with a simulated typist, including Gaussian "rough aim" (§4).
5. A real-typing study: 33 runs, an error-profile analysis that corrected the simulator, and a fitted personal error model (§6).
6. A negative result: a GPU-trained neural language model barely helps, and an analysis of why (§7).

2 Related Work

Spelling correction. Edit distance [2, 11] and the noisy-channel formulation [1, 9] are the classical foundation. The latter chooses the intended word w maximising $P(t \mid w)P(w)$ for typed string t. Real-word errors need context [12], and fast candidate generation is often done by symmetric deletion [5]. These systems model cognitive misspellings as much as motor slips, and correct one word at a time.

Decoding touch input. Soft keyboards combined key-press likelihoods with language models from early on [6]. Shape writing decodes gestures [10], and sentence-level decoding of very imprecise taps supports fast, accurate entry [16]. Our lattice decoder applies the same principle to discrete key events, where the only spatial evidence is *which* key fired.

Typing behaviour. Large keystroke datasets [3] and typo corpora [8] document how people mistype. We could not access these from our build environment, which is one reason we collected our own logs.

3 System

3.1 Formulation

For a typed chunk t with key times $\mathbf{k}$ and context c, the decoder seeks

$$\hat{w} = \arg\max_{w \in \mathcal{H}(t)} \; P(t, \mathbf{k} \mid w)\; P(w \mid c). \tag{1}$$

The hypothesis set $\mathcal{H}(t)$ contains:

- dictionary candidates;
- run-together segmentations (`thecat` → *the cat*);
- session-learned words;
- a *keep* hypothesis: the typed string is exactly what was meant.

Scores are costs in nats ($-\ln p$). For an unknown string, the keep cost is $\kappa_0 + \kappa_1 \min(|t|, 10)$. It makes long random strings unlikely as intentional words, while not penalising long genuine technical terms.

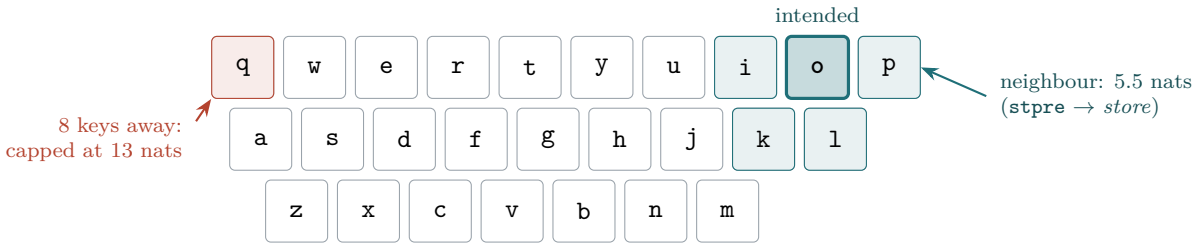


Figure 3: The error model on a row-staggered QWERTY layout. Hitting a neighbour of the intended key (teal) is a common slip; a far key (red) is very unlikely. Costs are $-\ln p$: every 2.3 nats is a factor of 10.

Table 1: Error-model costs in nats: default, and the profile fitted to the study participant (§6).

Operation	Default	Fitted
Substitution, adjacent key	5.5	5.5
per extra key of distance	+3.0	+2.0
Extra key: repeat / adjacent / other	5.5 / 6.0 / 11.0	5.5 / 6.0 / 7.0
Missing letter: doubled / other	4.0 / 6.5	4.0 / 4.5
Transposition (fast / normal / slow)	3.0 / 5.0 / 6.5	4.5 (no timing)

3.2 Error model and candidate search

Figure 3 illustrates the error model. A substitution costs more the farther apart the two keys are on a row-staggered QWERTY layout. An extra key is cheap when it repeats or neighbours an adjacent typed key. Dropping a doubled letter or an apostrophe is cheap. A transposition can be made cheaper when the two keys came within 45 ms (roll-over). Table 1 lists the costs.

Candidates come from running this weighted Damerau–Levenshtein recurrence directly over a trie of about 86,000 words, pruning any branch whose cheapest partial alignment exceeds a bound. The bound grows with word length, so long words can absorb more slips. A per-thread cache avoids repeating the search when a chunk is re-examined.

3.3 Language model

Unigram frequencies combine Google Books counts via SymSpell [5, 13] with `wordfreq` [15], filtered for web misspellings (§8). Bigram and trigram counts come from all 1.4 billion words of English Wiki-40B [7], plus conversational text weighted up. The store holds 11.5M word pairs and 13M triples (62 MB). The context cost is

$$P(w \mid u, v) = \lambda_3 \frac{c(u, v, w)}{c(u, v)} + (1 - \lambda_3)\big[\lambda P_2(w \mid v) + (1 - \lambda) P_1(w)\big], \tag{2}$$

where $\lambda_3 = 0.7\, c(u, v)/(c(u, v) + 5)$ and $\lambda = 0.6$. Here P_2 takes the larger of the Google Books and corpus estimates, since each list misses pairs the other has.

3.4 Sliding-window lattice decoder

Deciding each word alone throws away information. QuipKeys re-decodes the last four chunks as a lattice (Figure 4):

- **Single readings** for each chunk: keep, the top spelling candidates, splits, and learned words.
- **Boundary readings** for each adjacent pair: a merge (`meetin g`), and the space moved one letter either way (`hellon ice`).

A beam search over states of the last two words [14] finds the best path under the trigram model.

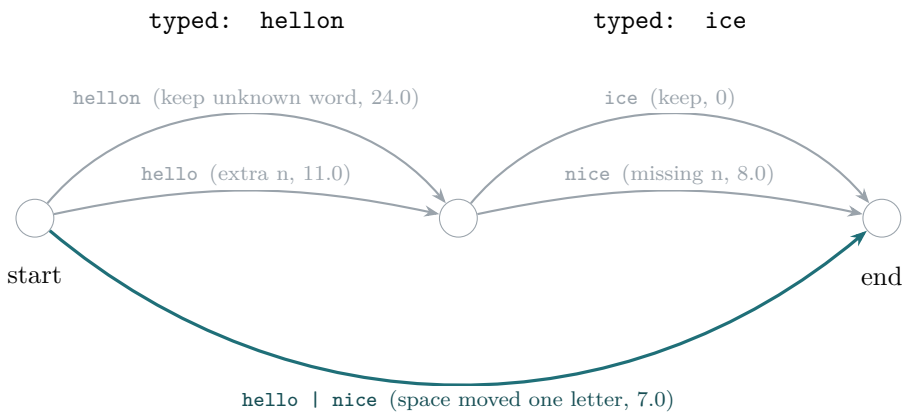


Figure 4: A lattice over two typed chunks. Grey arcs are single-chunk readings, and the teal arc is a boundary reading that moves the space. Each arc's error cost (nats) is shown; the language model adds a cost for every word along the path. Moving the space (7.0) explains the typing far more cheaply than two separate slips (11.0 + 8.0), and the single-chunk path through *hello* and *nice* produces the same text, adding to its probability.

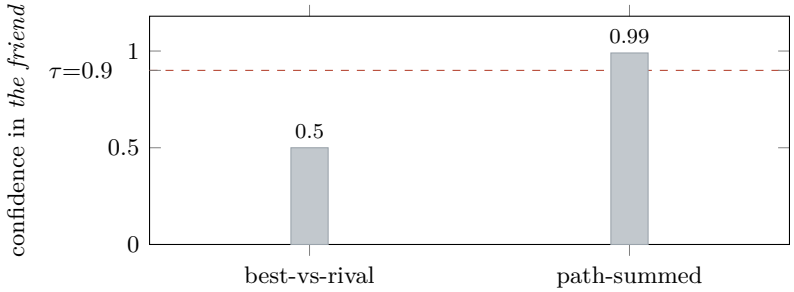


Figure 5: Why confidence must sum over paths, for `th efriend`. Two paths produce *the friend*. Comparing the best path with its best rival splits the evidence about 50/50, below the threshold τ; summing paths by output text gives about 0.99. Values are illustrative of the measured behaviour.

Path-summed confidence. A change at chunk k is applied only if its posterior clears a threshold:

$$P(\text{text}_k = y) = \frac{\sum_{\pi:\,\text{text}_k(\pi)=y} e^{-S(\pi)}}{\sum_{\pi} e^{-S(\pi)}} \;\geq\; \tau, \tag{3}$$

computed with a beam-pruned forward pass. The thresholds are $\tau = 0.9$ for non-words, and 0.98 (plus a 1.5-nat cost) for changing a real word. Summing over paths matters. An earlier version compared the best path against the best competing path, and refused `th efriend` → *the friend*, because two different paths produce that text (move the space, or drop the stray `e`) and each made the other look like a rival (Figure 5).

3.5 Safety mechanisms

- **Protected tokens** are never touched: digits, `snake_case`, `camelCase`, paths, URLs, e-mail addresses, flags (`-force`), ALL-CAPS acronyms, dialect elisions (`countin'`) and abbreviations (`Mr.`).
- **Names.** An unknown capitalised word mid-sentence needs 99.9% confidence. It may be fixed toward a known proper noun only if the slip is a single clean keyboard slip, not in the first letter. This separates `Calicornia` → *California* from `Colman` → *Coleman*: names differ by spelling-variant patterns, typos by keyboard patterns. Words normally capitalised, learned from Wikipedia casing, are restored (`london` → *London*).

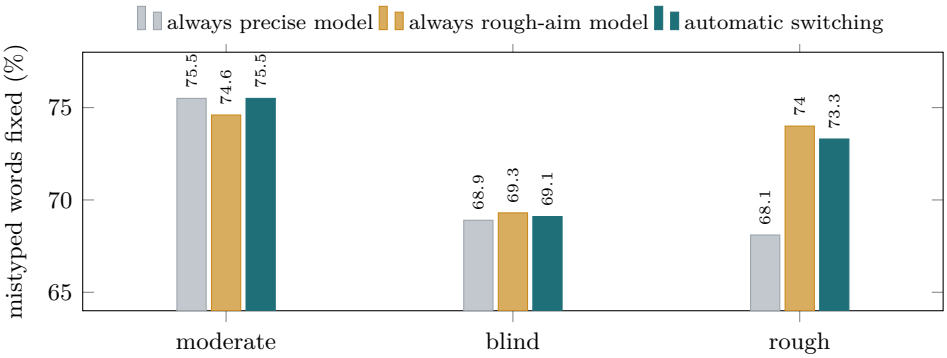


Figure 6: Switching error models by measured sloppiness (whole documents, development data). The switch keeps careful typing on the precise model and gives rough typing most of the rough-aim model's 5-point gain.

- **Morphology guard.** A known stem plus a regular affix (`freewheelers`) is treated as a real word.
- **Hysteresis.** A word changes at most once. The only exception is a boundary repair, which may override an earlier spelling fix.
- **Undo and learning.** Undoing a correction protects the word and learns it. So does typing a word identically twice. Later typos of learned words are fixed toward them (`kubetcl` → *kubectl*).
- **English variety.** When a typo is corrected and both UK and US forms exist (1,115 pairs), the preferred variety (UK by default) wins. Correctly typed words are never changed.

3.6 Errors beyond letters

- **Spacing.** Missing spaces are handled by segmentation, with exact pieces or one fuzzy piece. Premature and shifted spaces are handled by boundary readings. Merging two real words requires 6 extra nats of evidence, because merges are how compounds get mangled (`soy beans`).
- **Hand shift.** A word typed one key over (`jr;;p` → *hello*) is detected by remapping it.
- **Shift slips.** Patterns such as `THe`, `hTe` and `uSre` → *Sure* are recognised: a capital in second position means the first two keys rolled over.
- **Accidental Caps Lock.** Three inverted-case words in a row (`w NEED TO`) switch on case flipping, including mistimed Shift under Caps Lock (`thANSK` → *Thanks*). Real acronyms do not trigger it.

3.7 Adapting to sloppiness

Rough aim needs a different error model: adjacent slips cheap, distant keys steep. QuipKeys estimates sloppiness online as the share of the last 50 words typed as non-words. It switches to the rough-aim model above 45% and back below 35% (Figure 6).

4 Evaluation Methodology

Text. Clean sentences come from the Brown corpus [4], which is independent of all the engine's data sources. There are 1,500 development sentences for all tuning, 1,500 held-out test sentences for reporting, and 60 whole documents for document-level effects.

Table 2: Simulated typing levels.

Level	Letters mistyped	Words mistyped	
light	1.5%	7%	careful typist
moderate	3%	14%	
messy	6%	26%	
blind	10%	38%	not looking at the keys
rough	Gaussian aim	49%	the north star

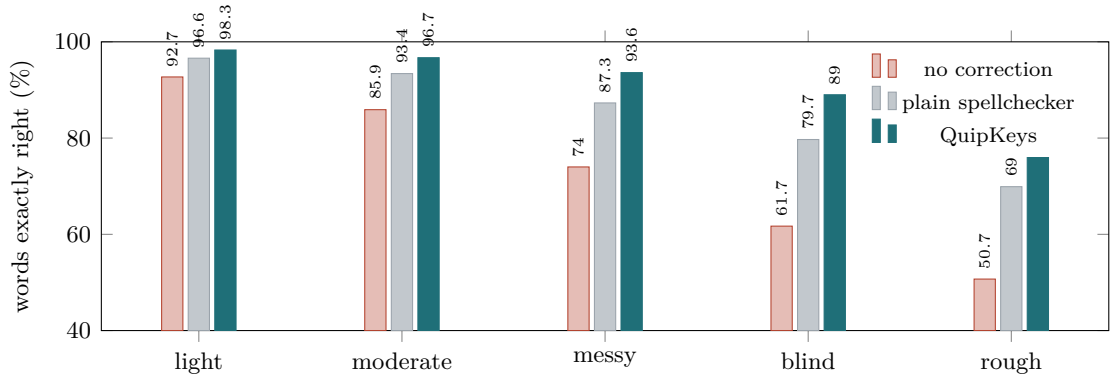


Figure 7: Word accuracy on held-out simulated typing. The sloppier the typing, the larger QuipKeys's margin over a spellchecker.

Simulated typist. A simulated typist types the clean text. The discrete-slip model mixes:

- adjacent-key substitutions;
- transpositions, with short inter-key intervals (which also occur on 6% of correct pairs, so timing is a hint and not a give-away);
- omissions, fat-finger presses and double taps;
- missing, premature and shifted spaces, and whole-hand shifts.

The *rough-aim* model lands every press at a Gaussian offset around the intended key ($\sigma = 0.30$ key widths horizontally, 0.24 vertically), with roll-overs and drops on top. Table 2 lists the levels.

Metrics. For mistyped words we report the share *fixed*, changed to the *wrong* word, or *left* as typed. For correctly typed words we report the *false-correction* rate, the number that decides whether people keep a corrector on. Word accuracy is the share of words output exactly right. Latency is per word on one CPU core.

Baseline. *plain* is a frequency-only spellchecker built from the same parts, without geometry, timing, context, splits or the lattice. We could not run operating-system autocorrect in our environment (§9).

5 Results on Simulated Typing

Figure 7 and Table 3 give the held-out results. QuipKeys fixes 24–34 points more mistyped words than the baseline at every level, with false corrections of about one in 750 correctly typed words.

Ablations. Table 4 switches components off one at a time.

- **Keyboard geometry** is the most valuable single component: 5–6 points, and 17 under rough aim.

Table 3: Held-out simulated typing (1,500 Brown test sentences per level).

Level	System	Fixed	Wrong	False corr.	Word acc.	p50 / p99 (ms)
light	no correction	–	–	–	92.72%	–
	plain	55.0%	2.5%	0.095%	96.64%	0.3 / 2.8
	QuipKeys	78.7%	2.0%	0.135%	98.33%	0.5 / 3.1
moderate	no correction	–	–	–	85.92%	–
	plain	53.5%	3.3%	0.078%	93.39%	0.4 / 3.1
	QuipKeys	77.3%	2.4%	0.132%	96.69%	0.6 / 4.0
messy	no correction	–	–	–	73.98%	–
	plain	51.3%	3.9%	0.080%	87.27%	0.6 / 3.8
	QuipKeys	75.5%	3.3%	0.109%	93.55%	0.8 / 5.3
blind	no correction	–	–	–	61.74%	–
	plain	46.9%	4.8%	0.076%	79.65%	0.8 / 4.3
	QuipKeys	71.5%	4.1%	0.125%	89.03%	1.0 / 8.3
rough	no correction	–	–	–	50.71%	–
	plain	39.1%	5.6%	0.076%	69.92%	0.9 / 4.7
	QuipKeys	72.9%	4.1%	0.135%	86.55%	1.2 / 9.9

Table 4: Ablations: share of mistyped words fixed (held-out simulated typing).

Configuration	moderate	messy	blind	rough
QuipKeys (full)	77.3%	75.5%	71.5%	72.9%
word-by-word instead of lattice	77.1%	74.5%	70.2%	70.7%
no keyboard geometry	72.3%	70.5%	65.2%	56.0%
pairs only (no trigrams)	74.0%	72.8%	68.7%	69.9%
no boundary repair	74.4%	72.2%	68.6%	70.5%
never switch to rough-aim model	77.3%	75.5%	71.1%	69.4%
plain (frequency-only)	53.5%	51.3%	46.9%	39.1%

- **Trigram context** and **boundary repair** each add about 3 points.
- **The lattice decoder** adds 0.2–2 points.
- **The rough-aim error model** adds 3.5 points on rough typing.

6 Real Typing

Protocol. One participant, the author, typed 33 thirty-word passages of everyday sentences in a browser-based Typing Lab, which logged every keystroke with millisecond timing and applied no correction. All runs used the north-star condition: as fast as possible, not looking, backspace disabled. Most hid the typed text, because seeing it prompted the participant to go back and fix. Five early runs were typed with Caps Lock on by accident. Logs were replayed through the engine and scored on words, ignoring case and punctuation, since the participant mostly omitted punctuation.

Accuracy. Words typed exactly right rose from 75.4% to 91.5% (Figure 8). Every run improved, and three reached 96–100%. At the median raw speed of 85 WPM, that is roughly 64 correct words per minute uncorrected versus 78 with QuipKeys. On the 21 runs typed after the personal error model was fitted, the engine scored 92.4%, or 93.0% with the fitted model.

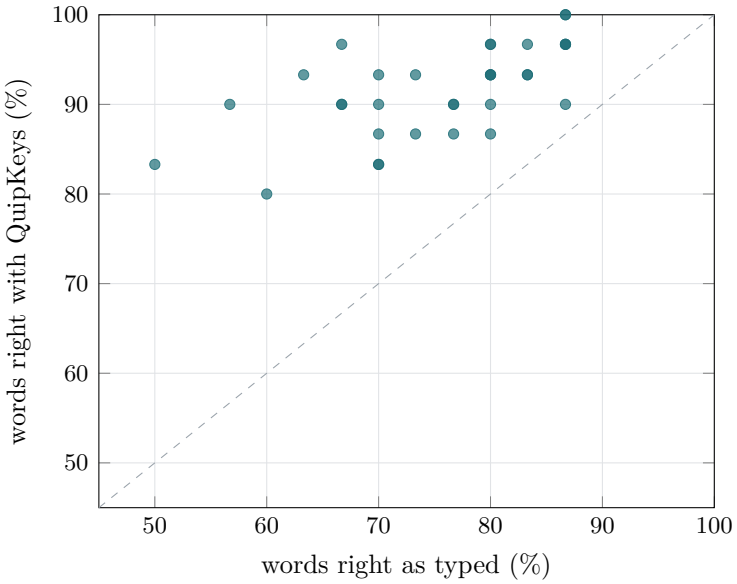


Figure 8: Real typing, one point per 30-word run (33 runs, 75–96 WPM, not looking, no backspace). Every run lies above the diagonal.

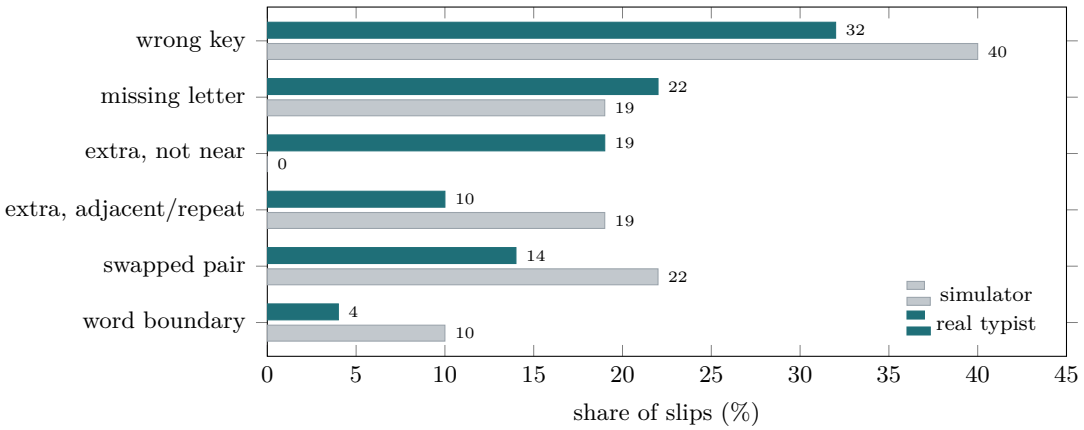


Figure 9: Where the slips came from: real typist (33 runs, 3,944 letters, 8.8% mistyped) versus the discrete-slip simulator. The simulator had no extra letters away from neighbouring keys, the real typist's most distinctive slip.

The real error profile. Figure 9 compares the participant's 346 slips with the simulator's assumptions. Two assumptions were wrong:

1. Most extra letters are *not* neighbours of the keys around them (66 of 100).
2. Swapped pairs happen at normal pace (about 100–150 ms between keys), not as sub-45 ms roll-overs.

Ablations on real typing. Figure 10 shows the decisive difference between simulated and real typing. Keyboard geometry, the most valuable component in simulation, contributed nothing on real typing. Context and boundary repair carried the gains, and a frequency-only spellchecker reached 84.7%.

Fitted error model. Re-estimating the error costs from the real profile (Table 1) gained 1.7 points on the 12 runs it was fitted to, and 0.6 points on the 21 runs typed afterwards. The gain is small but it generalises. It is opt-in for now, because cheaper extra and missing letters widen the

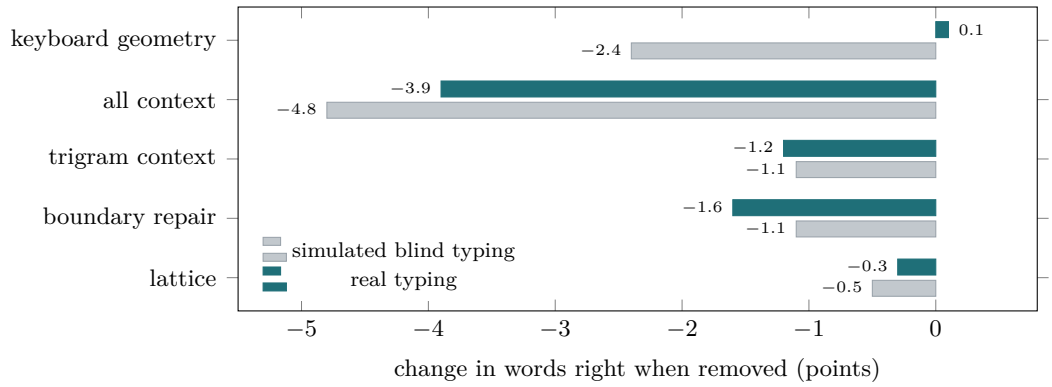


Figure 10: Removing each component, simulated blind typing versus real typing (word accuracy). On real typing, removing geometry changes nothing; context matters most.

search and slow the worst case.

7 A Neural Language Model

Half of the words still wrong on real typing are *real-word* slips: `ship` for *shop*, `our`/`out`, `nice` for *nine*, `quite`/`quiet`, `fat`/`far`. Trigram context cannot overrule these safely, so we trained a neural language model.

Model and training. The model is a 6-layer, 6-head decoder-only transformer with 384 dimensions and 23.3M parameters. Its word-level vocabulary is the engine's 32,768 most frequent words plus unknown-word and boundary tokens, with 64-word context and tied input/output embeddings. Training text was 32 Wiki-40B files plus conversational text weighted up: 398.9M tokens, tokenised with the engine's own tokeniser. Training ran for 110,000 steps (batch 64), about 3 hours on an RTX 2060 at about 55k tokens/s, reaching validation perplexity 59. A CPU proof of concept (4.6M parameters, 1 hour) had reached about 200. The exported weights are int8 with per-row scales, and inference is pure Rust with an incremental key-value cache.

Two ways to use it.

1. As a *left-context re-ranker*: each chunk's readings get $\alpha(\text{lse} - \text{logit})$ added, using the model's next-word logits given everything before the chunk.
2. As a *look-ahead* for real words: two words after a real word is typed, each one-slip real-word alternative is tried in place, and the model scores the alternative plus the following words with a full softmax.

Results. Figure 11 shows the outcome. On real typing it gave at most +0.3 points. The look-ahead changed no outputs at all. On simulated typing the re-ranker doubled real-word fixes (8% to 17%), but cost about a point overall, raised false corrections, and was 50× slower (18 ms median per word, unoptimised).

Why it did not help. We scored each candidate together with the words after it. The model's preferences were usually right, but the evidence was weak or too far away (Table 5):

- *our* → *out* gets 13.6 nats, and is fixed.
- *fat* → *far* gets 3.6 nats, and *quite* → *quiet* 2.1. That is not enough to overcome the roughly 7-nat prior against a one-slip real-word error (a neighbouring-key slip that lands on another valid word).

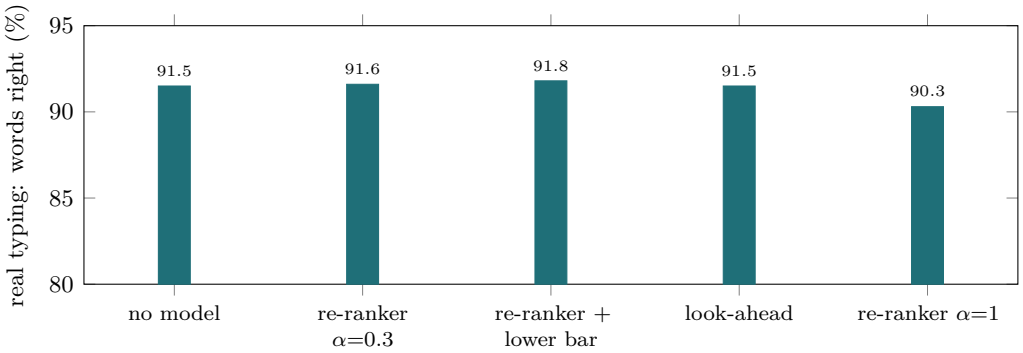


Figure 11: The 23M-parameter neural language model on real typing (33 runs). At best it adds 0.3 points; at full weight it hurts.

Table 5: Neural evidence for real-word alternatives: log-probability of (candidate + following words) given the preceding words, as the advantage in nats of the intended word over the typed one.

Context	Typed → intended	Evidence	Outcome
thanks for helping _ with the	our → out	+13.6	fixed
ate _ too much popcorn	fat → far	+3.6	below the bar
it was nice and _ in the	quite → quiet	+2.1	below the bar
go to the _ before it closes	ship → shop	+0.0	no preference

- For *ship* → *shop* the model has no preference: the disambiguating word (*closes*) is three words away.

In the participant's real typing, the words after a real-word slip are often misspelled themselves, which weakens the evidence further. Lowering the prior to let 2–4 nats through would also flip correct words, the failure the system exists to avoid.

Interpretation. The remaining errors are short of *information*, not intelligence. A bigger model would sharpen preferences such as *far*/*fat*, but cannot invent evidence that the keystrokes and nearby words do not contain (§10).

8 What Remains, and Lessons

Of the 81 words still wrong across the 33 real runs (Figure 12):

- 49% are real-word slips (§7);
- 48% are non-words with two or more slips, or genuinely ambiguous readings (`asn`: *and* or *an*? `thei`: *this*, *their* or *the*?);
- 2% are spacing errors.

Negative results.

- **4-grams.** 6.6M 4-grams from the same 1.4B words gave no gain and more false corrections.
- **A learned decision gate.** A logistic model over 18 decision features improved its training objective, but raised false corrections on the benchmark. It was not adopted.
- **A document word cache** reinforced uncorrected slips. It was replaced by learning only words typed identically twice, which adds 0.1–0.3 points on documents.

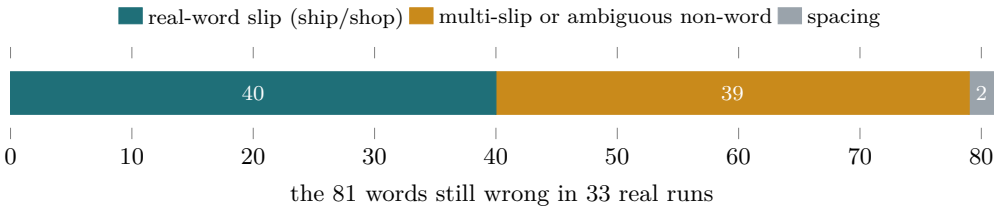


Figure 12: What is left. Half the remaining errors are real-word slips, which need evidence the context rarely provides; half are badly garbled or genuinely ambiguous.

Dictionary hygiene. A rule admitting UK/US variant pairs (`enrolment`/`enrollment`) also admitted 364 web misspellings (`actualy`, `realy`, `allready`) as words. Real variants are within about 10× of each other in frequency; misspellings are 300–3000× rarer than their word. A frequency-ratio filter separates them. The participant's own typing (`actyaly`) exposed the bug; the simulator never could.

9 Limitations

- **One participant**, who is also the author. The held-out split guards against fitting to particular runs, not to a particular person.
- **No comparison with operating-system autocorrect.** The baseline is our own frequency-only spellchecker.
- **The key behavioural question is unmeasured:** does fast, sloppy typing with QuipKeys beat careful typing with backspace? The participant's careful baseline was not recorded.
- **The simulator is partly wrong** for this typist (Figure 9), so simulated results should be read with that in mind.
- **The evaluation text** is 1960s American prose, while the engine prefers UK spellings.
- **No operating-system integration yet.**

10 Future Work: More Information, Not More Intelligence

The analysis in §7 points to supplying new evidence rather than larger models:

- **Key-up timing.** Physical keyboards report key-up as well as key-down. Extra letters are this typist's most distinctive slip, and accidental brushes may show unusually short hold times or overlap with neighbouring presses. That signal is free, and invisible in text.
- **Personal text.** A language model adapted to the user's own writing (their phrases, names and topics) gives exactly the context that general text lacks.
- **Interaction instead of guessing.** For an uncertain real word, underline it and let the user accept a suggestion with one key. A one-key fix still beats retyping, and nothing is silently changed.
- **Online personal error models,** learned from accepted and undone corrections.
- **A proper study:** careful typing, sloppy typing with QuipKeys, and sloppy typing with built-in autocorrect, with more participants, including typists with motor impairments, for whom imprecision is not a choice.

11 Conclusion

Phone-style decoding transfers to physical keyboards. On real blind typing at 85 WPM, QuipKeys raised exact-word accuracy from 75% to over 91% while rarely touching correct text. Real data was decisive throughout. It corrected the simulator, found a dictionary bug, and showed that context, not geometry, carries the gains for this typist. A neural language model trained for the remaining real-word slips confirmed that those errors are limited by missing information. The next gains will come from new signals (timing, personal text) and from interaction design, not from more model capacity.

Acknowledgements. The engine, benchmark, data-collection tools and this report were developed with the help of Claude, an AI assistant by Anthropic.

References

[1] Eric Brill and Robert C. Moore. An improved error model for noisy channel spelling correction. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 286–293, 2000.

[2] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.

[3] Vivek Dhakal, Anna Maria Feit, Per Ola Kristensson, and Antti Oulasvirta. Observations on typing from 136 million keystrokes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018.

[4] W. Nelson Francis and Henry Kučera. *Brown Corpus Manual*. Department of Linguistics, Brown University, 1979.

[5] Wolf Garbe. SymSpell: Symmetric delete spelling correction algorithm, 2012. Software, `https://github.com/wolfgarbe/SymSpell`.

[6] Joshua Goodman, Gina Venolia, Keith Steury, and Chauncey Parker. Language modeling for soft keyboards. In *Proceedings of the 7th International Conference on Intelligent User Interfaces (IUI)*, pages 194–195, 2002.

[7] Mandy Guo, Zihang Dai, Denny Vrandečić, and Rami Al-Rfou. Wiki-40B: Multilingual language model dataset. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, 2020.

[8] Masato Hagiwara and Masato Mita. GitHub typo corpus: A large-scale multilingual dataset of misspellings and grammatical errors. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, 2020.

[9] Mark D. Kernighan, Kenneth W. Church, and William A. Gale. A spelling correction program based on a noisy channel model. In *Proceedings of the 13th International Conference on Computational Linguistics (COLING)*, 1990.

[10] Per Ola Kristensson and Shumin Zhai. SHARK^2: A large vocabulary shorthand writing system for pen-based computers. In *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 43–52, 2004.

[11] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[12] Eric Mays, Fred J. Damerau, and Robert L. Mercer. Context based spelling correction. *Information Processing & Management*, 27(5):517–522, 1991.

[13] Jean-Baptiste Michel et al. Quantitative analysis of culture using millions of digitized books. *Science*, 331(6014):176–182, 2011.

[14] Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[15] Robyn Speer, Joshua Chin, Andrew Lin, Sara Jewett, and Lance Nathan. LuminosoInsight/wordfreq: v2.2, 2018. Zenodo, doi:10.5281/zenodo.1443582.

[16] Keith Vertanen, Haythem Memmi, Justin Emge, Shyam Reyal, and Per Ola Kristensson. VelociTap: Investigating fast mobile text entry using sentence-based decoding of touchscreen keyboard input. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pages 659–668, 2015.